

# Tech Mock Interview

## AI-Driven Mock Interview Platform — Solution Design

|          |                                                                                                          |
|----------|----------------------------------------------------------------------------------------------------------|
| Version  | 1.0                                                                                                      |
| Date     | 9 July 2026                                                                                              |
| Status   | Design proposal (not yet built)                                                                          |
| Platform | Reuses the Focus Forge delivery platform: Spring Boot + React + Postgres/pgvector + OpenAI + Kafka + k3s |

# 1. Vision & Goals

Tech Mock Interview lets a candidate pick any technology, take a realistic timed interview across five question categories, and receive an objective score with actionable improvement advice — no human interviewer required.

- Question sets for **any technology** with zero code changes (data + prompts only).
- Five categories per interview: **Basics, Intermediate, Hard, Scenario-based, Company-style** questions.
- Answers written in a web form, one question per card, with timers and autosave.
- **Explainable scoring**: per-question rubric scores, missed points, model answers, category radar and an overall grade.
- **Personalised advice**: strengths, gaps, a 7-day study plan and progress tracking over repeated attempts.

# 2. User Journey

```
[Login] -> [Home: choose Technology + optional Target Company + Experience level]
-> [Interview]
    Sec 1 Basics(5)  Sec 2 Intermediate(5)  Sec 3 Hard(5)
    Sec 4 Scenario(3)  Sec 5 Company-style(3)
    one question per card | textarea | per-question timer | skip allowed | autosave 10s
-> [Submit] -> evaluating... (async 20-40s)
-> [Result] overall 0-100 + grade | radar per category |
    per-question: score /10, missed points, feedback, model answer |
    advice: 3 strengths, 3 gaps, 7-day study plan, resources
-> [History] past interviews, score trend, weak-topic tracker
```

# 3. High-Level Architecture

```
React SPA (Vite + shadcn/ui)
|
Traefik Ingress (same-origin /api/v1 -- proven Focus Forge pattern)
|
Spring Boot API (interview-service)
|-- Question Engine -----> OpenAI (Spring AI, structured output)
|   |
|   v
|   PostgreSQL + pgvector <-----+ question-bank cache, dedup via
|   (users, interviews,            embeddings, RAG grounding
|   question bank, results)
|
|-- Evaluation Engine (async worker) --> OpenAI (LLM-as-judge, rubric)
|
+-- Kafka topic mock-interview.events (analytics, streaks, digests)
```

| Decision          | Choice                                                 | Why                                                  |
|-------------------|--------------------------------------------------------|------------------------------------------------------|
| Question source   | Hybrid: curated DB bank + LLM generation on-stash heap | On-stash heap; generated sets are stored and reused  |
| Evaluation        | LLM-as-judge, fixed rubric, structured JSON            | Explainable, consistent, per-criterion scores        |
| Evaluation timing | Asynchronous (submit returns; UI polls)                | 16-21 judgments take 20-40s; never block the UI      |
| Company questions | Interviewer personas + curated company packages        | Realistic style without reproducing leaked questions |
| New technology    | A data row, not a deployment                           | Zero-code extensibility                              |

## 4. Question Engine (LLD)

- POST /interviews {technology, company?, experienceLevel} starts a session.
- Cache lookup: pgvector similarity search over prior question sets for (tech, level, company); a fresh set not yet seen by this user is reused as-is.
- Cache miss: one structured-output LLM call per category, run in parallel. The company category adds a persona block ('You are an interviewer at {company}; match their style').
- Output schema per question: {text, topicTag, idealAnswerPoints[], difficulty}. The idealAnswerPoints become the scoring rubric later - candidates are judged against points fixed at generation time, not at judge time.
- Anti-repeat: per-user question history feeds an 'avoid these recent topics' instruction into the generation prompt.

## 5. Evaluation & Scoring (LLD)

Each answer is judged by a parallel LLM call (temperature 0) against four weighted criteria; missed points are diffed against the stored idealAnswerPoints.

| Criterion            | Weight | Measures                                  |
|----------------------|--------|-------------------------------------------|
| Correctness          | 40%    | Technical accuracy vs ideal answer points |
| Depth                | 25%    | Internals, trade-offs, edge cases         |
| Clarity              | 20%    | Structure and communication quality       |
| Real-world grounding | 15%    | Examples, practical experience signals    |

Category score = mean of its questions. Overall = weighted mean (Basics 15%, Intermediate 20%, Hard 25%, Scenario 25%, Company 15%). Grade bands: 85+ Interview-ready, 70+ Strong, 50+ Developing, below 50 Foundations. A final career-coach LLM call produces 3 strengths, 3 gaps, a 7-day study plan and resources.

- **Guardrails:** empty/'idk' answers score 0 without an LLM call; off-topic paste detection via answer-vs-question embedding similarity; answer length caps; candidate text is wrapped as data in the judge prompt to defuse prompt injection.

## 6. Data Model

```
technology(id, name, icon, active)
company_pack(id, name, style_prompt, active)
question_set(id, technology_id, company_id?, level, created_at, embedding)
question(id, set_id, category ENUM(BASIC, INTERMEDIATE, HARD, SCENARIO, COMPANY),
  text, topic_tag, ideal_answer_points JSONB, position)
interview(id, user_id, set_id, status ENUM(IN_PROGRESS, SUBMITTED, EVALUATED),
  started_at, submitted_at, overall_score, category_scores JSONB)
answer(id, interview_id, question_id, text, duration_sec,
  score, criteria JSONB, missed_points JSONB, feedback, model_answer)
advice(interview_id, strengths JSONB, gaps JSONB, study_plan JSONB)
user_topic_stats(user_id, topic_tag, attempts, avg_score)
```

## 7. API Surface (context path /api/v1)

| Method | Endpoint      | Purpose                      |
|--------|---------------|------------------------------|
| GET    | /technologies | List selectable technologies |
| GET    | /companies    | List company packs           |

| Method | Endpoint                       | Purpose                                          |
|--------|--------------------------------|--------------------------------------------------|
| POST   | /interviews                    | Start interview (generate or reuse question set) |
| GET    | /interviews/{id}               | Fetch questions section by section               |
| PUT    | /interviews/{id}/answers/{qId} | Autosave an answer                               |
| POST   | /interviews/{id}/submit        | Freeze answers, enqueue evaluation               |
| GET    | /interviews/{id}/result        | Poll status; full result when EVALUATED          |
| GET    | /interviews/history            | Past interviews and progress series              |
| GET    | /me/weak-topics                | Spaced-repetition study suggestions              |

Authentication reuses the Focus Forge JWT + OAuth2 module.

## 8. Non-Functional Design

- **Cost control:** question-set caching (largest saver), gpt-4o-mini judging, token caps per answer, daily per-user interview quota.
- **Latency:** async evaluation queue; UI polls /result (upgrade path: WebSocket push - the infrastructure already exists in the cluster).
- **Scoring consistency:** temperature 0, fixed rubric, ideal answers frozen at generation time.
- **Observability:** actuator health + per-interview token/cost metrics emitted as events.

## 9. Deployment & Delivery

Identical to the proven Focus Forge pipeline: repos tech-mock-interview-backend / -frontend in Gitea; Gitea Actions on the existing host runner builds SHA-tagged images and imports them into k3d; manifests live in the existing gitops repo as a second ArgoCD application; ingress mockinterview.localtest.me:8081; the OpenAI key and Kafka cluster are reused; data in a separate database on the existing Postgres.

## 10. Extra Ideas (beyond the ask)

- **Voice round** - record answers, Whisper transcription, fluency judged too.
- **Adaptive difficulty** - later sections shift difficulty with the live score.
- **Coding questions** - embedded Monaco editor with hidden test cases.
- **Resume-aware interviews** - upload resume, RAG over it, questions about YOUR projects.
- **Mock panel mode** - two LLM personas (tech + HR) ask follow-ups on your previous answer.
- **HR/behavioural pack** - STAR-rubric scoring for behavioural rounds.
- **Leaderboard & streaks** - reuse Focus Forge gamification and Kafka events.
- **Shareable PDF report** - export the result page as an interview report.
- **Admin question marketplace** - curate generated questions into a permanent approved bank.
- **Multi-language** - answer and receive advice in the candidate's language.

## 11. Suggested Build Order

| Phase         | Scope                                                                                                            |
|---------------|------------------------------------------------------------------------------------------------------------------|
| MVP (≈1 week) | Single tech list, LLM generation without cache, synchronous evaluation, result screen with per-question feedback |
| v1            | Async evaluation, question-set caching, history, category radar chart                                            |
| v1.1          | Company packs, weak-topic tracker, Kafka events, PDF report export                                               |
| v2            | Adaptive difficulty, resume-aware questions, voice round                                                         |